



Onboard Visual Foundation Models for Mars Terrain Perception and Rover Navigation

Thupthai Chaiyadecha

Supervisor: **Dr Saurabh Upadhyay**

MSc Astronautics and Space Engineering

What this presentation covers





PART 1

Problem

Why Mars rovers need better onboard terrain understanding.

How a Mars rover actually gets driven today



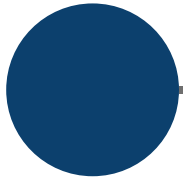
Curiosity rover on Mars

- 1 The rover photographs the ground
- 2 Send pictures to Earth
- 3 a team plans tomorrow's route by hand
- 4 the route is sent up and driven
- 5 the rover stops and waits

one loop takes a day, and it repeats

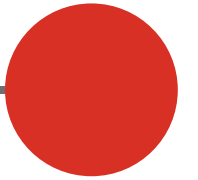
Why real-time control from Earth is impossible

3 to 22 minutes



Earth

one way, at the speed of light



Mars

Geometry alone cannot tell the rover what the ground is made of

1

Traditional hazard sensing focuses on geometry: slope, roughness, and step height.

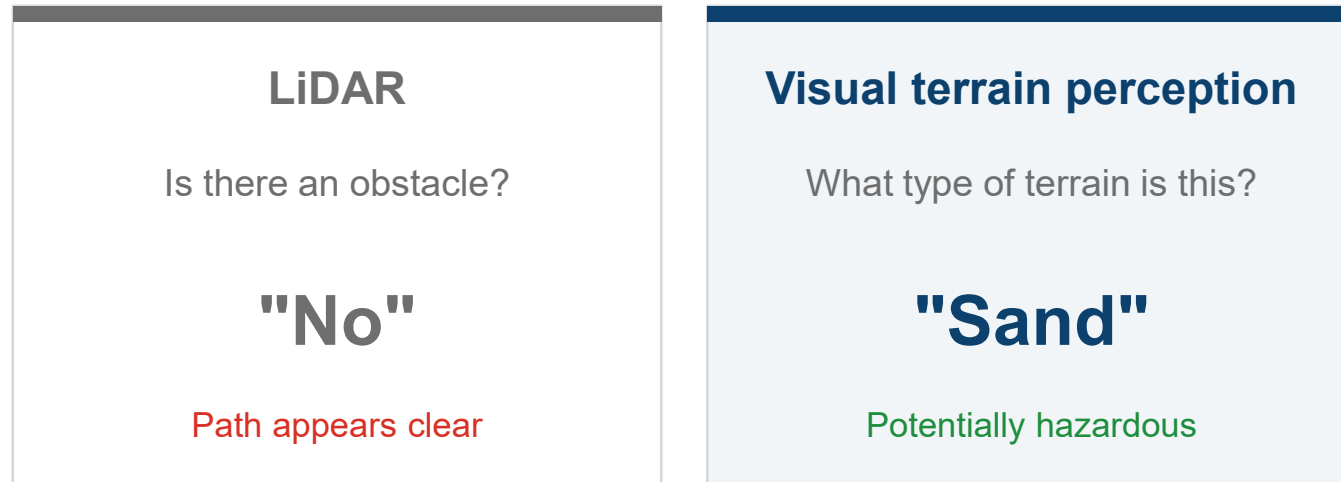
2

Sand and bedrock can present very similar geometry.

3

But they can behave very differently under the rover's wheels.

Why LiDAR alone cannot solve the problem



LiDAR measures distance and geometry.

A flat surface may be clear of obstacles but still be difficult to traverse.

Terrain type therefore requires information beyond geometry alone.

“Obstacle-free does not always mean traversable.”

The problem, stated plainly

Planning is Earth-led

High-level routes are planned on Earth, then executed autonomously onboard.

Geometry is not enough

Traditional hazard sensing focuses on shape, but terrain type can remain hidden.

So: limited and still risky

The rover can avoid obstacles, but may still misjudge the terrain itself.

Earth–Mars delay

limits direct human control

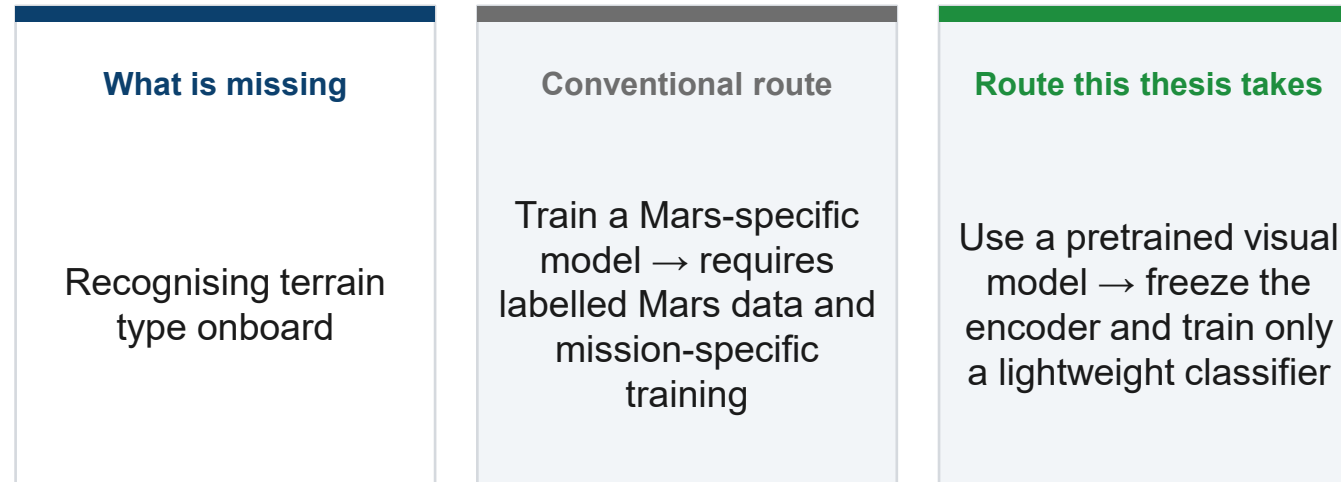
Geometry-based sensing
may miss terrain properties

Therefore

the rover needs better onboard terrain understanding

The core gap is onboard terrain understanding:
not just where the rover can drive, but what it is driving on.

The gap, and what could close it



Missing: recognising what the ground is made of, on board, by itself.

A dedicated Mars model requires labelled Mars data and mission-specific training.

So: take a pretrained visual model, keep its encoder frozen, and train only a lightweight classifier.



PART 2

Foundation models

What foundation models are, where DINOv2 fits, and how 22 models were compared.

A foundation model is pretrained on large-scale data, then reused for new tasks

	trained on	can do	who trains it
Ordinary model	one task-specific dataset	one specific task	task developer
Foundation model	large-scale, diverse data	many downstream tasks	pretrained by another organisation

An ordinary model is trained for one specific task.

A foundation model is pretrained on broad, large-scale data to learn reusable representations.

It can then be adapted or reused for tasks it was not specifically trained for.

The expensive pretraining is done once; downstream users reuse the learned representation instead of training the model from scratch.

Frozen means the pretrained encoder stays unchanged; only a small classifier is trained

	Fine-tuning	Frozen encoder (used here)
What changes	Model parameters are updated	Encoder weights stay fixed
What is trained	Part or all of the network	Only a small classifier
Compute	Higher	Low
Training time	Longer	Seconds to minutes

Fine-tuning updates parameters inside the pretrained model and usually requires more compute.

Here, the encoder is left completely unchanged.

Only a lightweight classifier is trained on top of its frozen features.

Nothing inside the encoder is updated at any point anywhere in this thesis.

Three model families, distinguished by how they were pretrained

Family	Learns from	Examples	Best result
Visual self-supervision	Images only; no captions or class labels	DINOv2, DINOv3, Franca	94.43% (ensemble)
Image-text pretraining	Image-caption pairs	CLIP, SigLIP2	87.80%
Task-specific supervision	Masks, depth targets, task-specific signals	SAM2, Depth Anything	75.30%

Visual self-supervision

Learns visual structure without captions or class labels.

Image-text pretraining

Learns to align visual features with language.

Task-specific supervision

Learns representations shaped toward a particular visual task.

DINOv2 learns from images alone: no captions, no class labels

Self-supervised. No captions, no class labels, no vocabulary of any kind.

It is not a vision-language model; it does not generate language descriptions.

It represents each image as a 384-dimensional visual feature vector.



no words go in, and no words come out

The feature vector captures visual similarity, so similar terrain tends to lie closer in feature space.

Why image-only pretraining can be an advantage on Mars

4 Mars classes	Typical language labels	Potential ambiguity for a language-based model
soil	soil / dirt / surface	broad/overlapping terminology
bedrock	rock	overlaps with “rock”
sand	sand / dust	broad/overlapping terminology
big rock	rock	overlaps with bedrock

Mars terrain classes are visually specific, but their everyday language labels can overlap.

For example, “bedrock” and “big rock” both contain the concept of rock.

Image-only self-supervision does not depend on those linguistic distinctions.

AI4Mars: the labelled Mars dataset used throughout this study

	Soil	Bedrock	Sand	Big rock
usable training images	1,000	1,000	1,000	108
in the 287-image test set	123	92	72	0

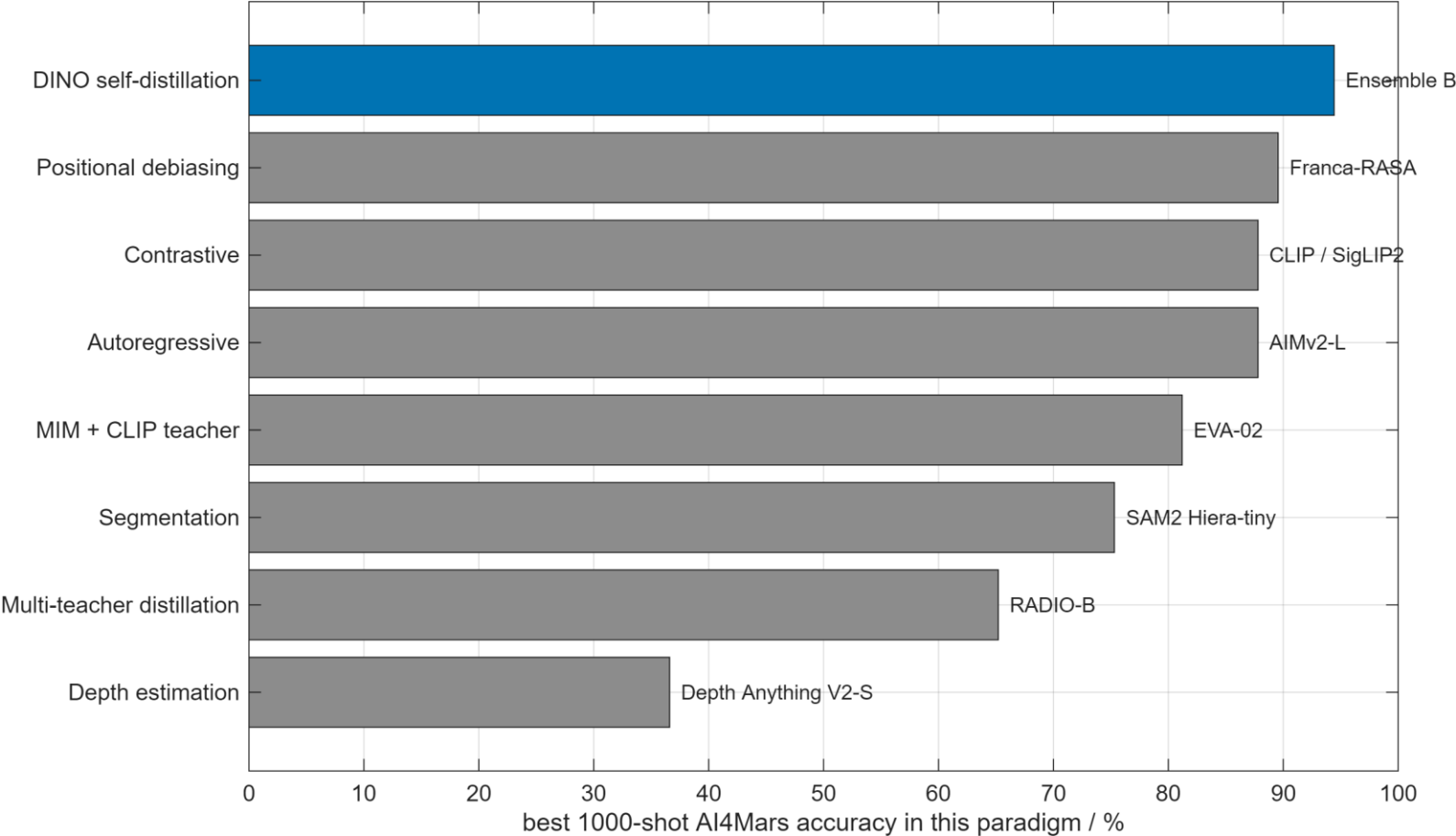
Real Curiosity camera images, labelled by volunteers, four terrain types.

Training images are used for the lightweight classifier; the held-out test set evaluates every model under the same protocol.

Big rock is severely underrepresented: only 108 usable training images, and none in the 287-image gold test set.

A 287-image test set, checked by planetary scientists, is held back from training throughout.

How 22 models were compared fairly



Same images, same classifier, same settings, same test set, every time.

For the single-encoder comparisons, the encoder is the only component that changes. Combined models are evaluated separately.

Only the lightweight classifier is trained on Mars labels; every encoder remains frozen.

Best AI4Mars result within each of the eight pretraining objectives.

The result: image-only self-supervision performs best on Mars terrain

94.43%

Best overall

Two frozen DINOv2
encoders combined

90.24%

Deployed model

DINOv2+reg ViT-S/14

94.43% only 2.24 points below the 96.67% supervised baseline



PART 3

ExoMy

The rover, its sensors, and how a camera frame becomes a driving command.

Hardware: the ExoMy rover



ESA's open-source, 3D-printed rover with six driven and steered wheels.

Raspberry Pi 4, 8 GB. No graphics card, no accelerator, four CPU cores.

Mast-mounted camera, 360° LiDAR, and IMU for tilt sensing.

Built and instrumented for this thesis. The deployed perception and control stack runs on a single Raspberry Pi 4.

Only the camera-based model identifies the terrain

Sensor	Identifies terrain	May veto	Influences route
Camera + DINOv2	yes	yes	yes
LiDAR	no	yes, inside 0.40 m	no
IMU	no	yes, tilt over 20 deg	no

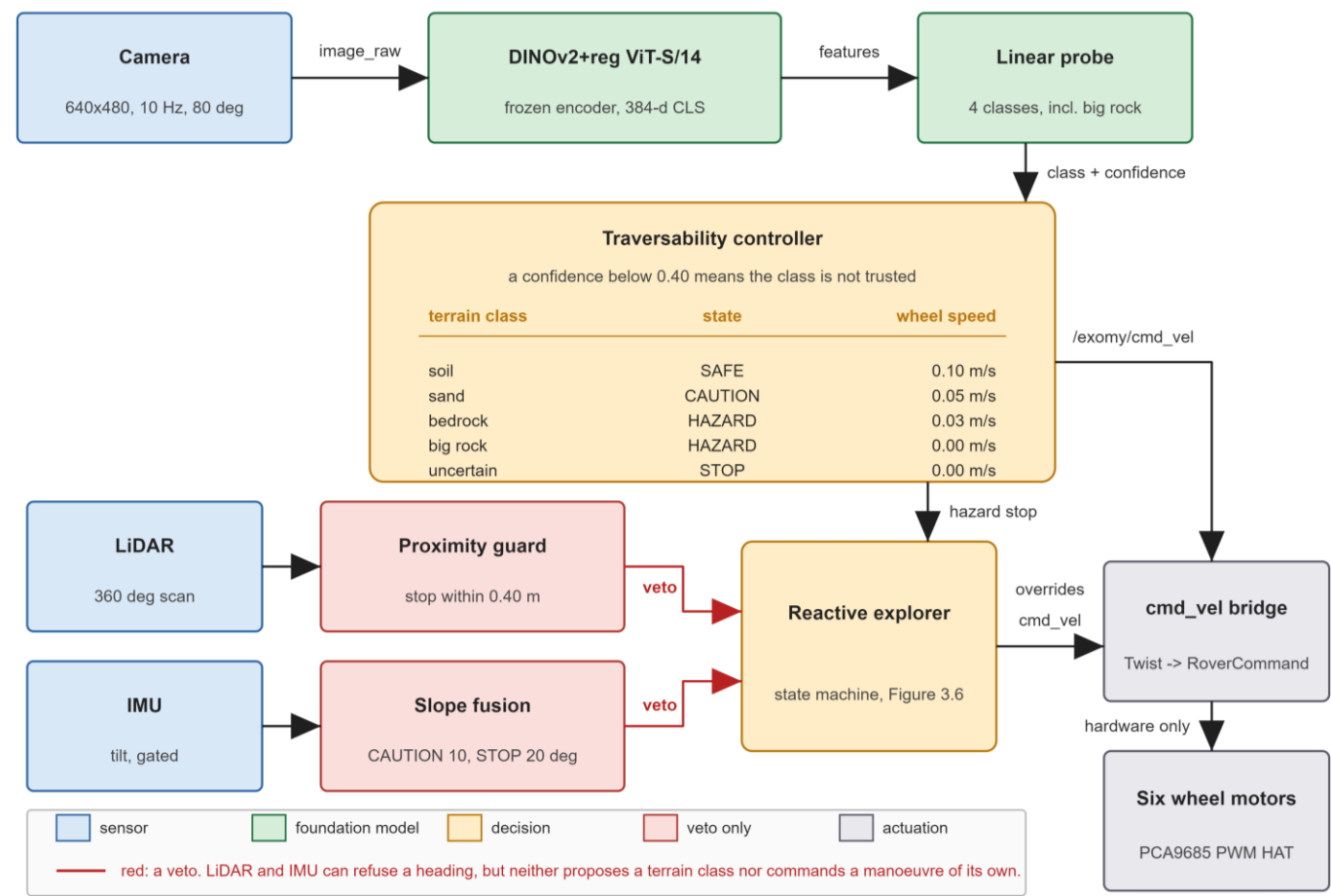
Camera: the only input the foundation model ever sees.

Laser scanner: may veto. It stops the rover inside 0.40 m, and nothing else.

Tilt sensor: may veto. Caution past 10 degrees, stop past 20.

LiDAR and IMU can veto unsafe motion, but neither identifies the terrain.

Perception to motion: terrain classification informs the rover's driving decision



Green is the frozen foundation model. Red arrows are safety vetoes.
Only the lightweight classifier is trained on Mars labels.

How the real hardware changed the speed policy

what the policy asks for		what the wheels actually did	
terrain class	speed asked for	motor command value	speed measured
soil	0.10 m/s	20	0.235-0.245 m/s
sand	0.05 m/s	25	0.235-0.245 m/s
bedrock	0.03 m/s	50	0.235-0.245 m/s
big rock	0.00 m/s	any non-zero	the same speed
uncertain	0.00 m/s	0	stopped

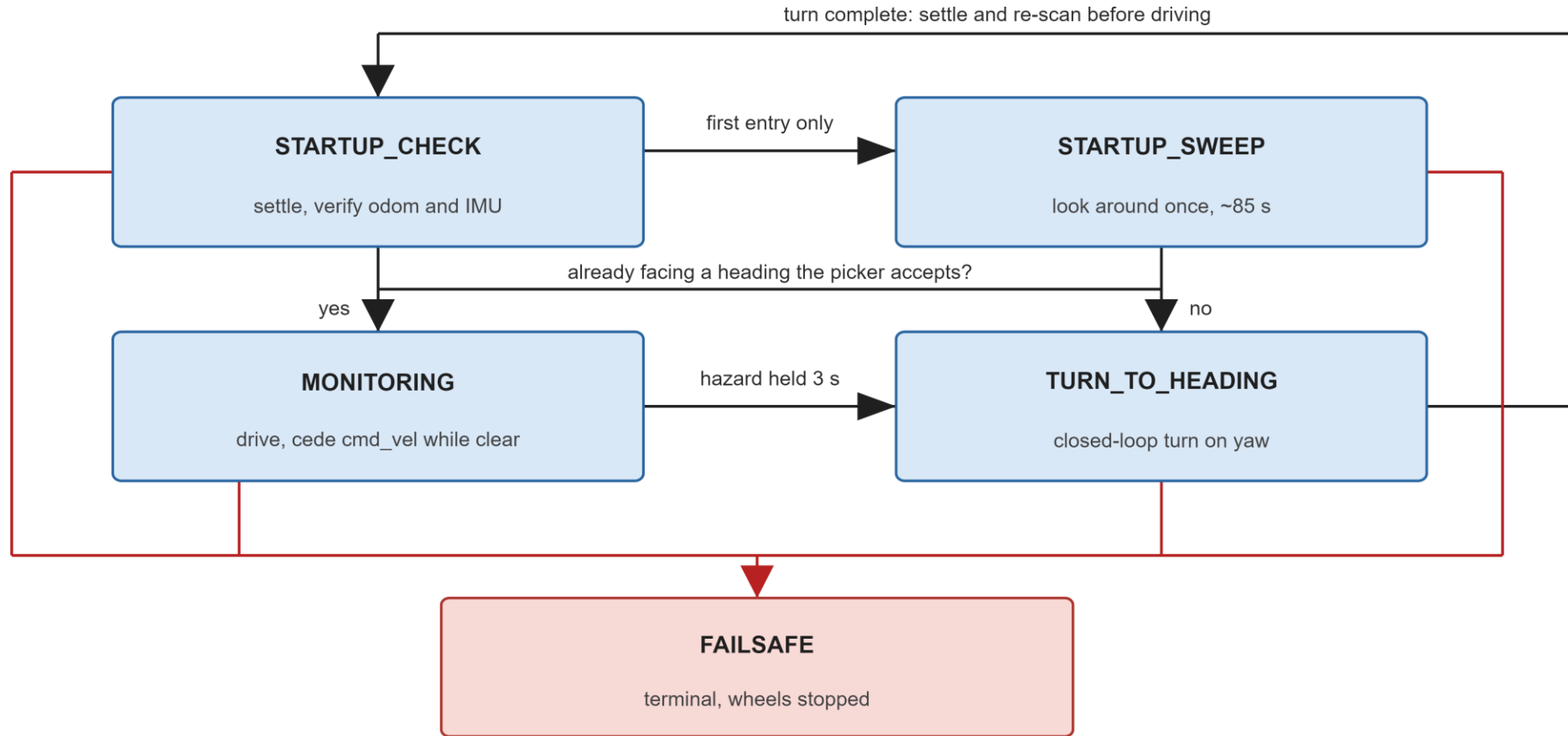
On paper: a different speed for each terrain class, 0.10 m/s down to a stop.

Measured on a 3 m course: three different commands, one identical speed.

The drive system saturates at low command values, so all non-zero commands produce nearly the same speed.

On this rover the policy is not four speeds. It is go, and stop.

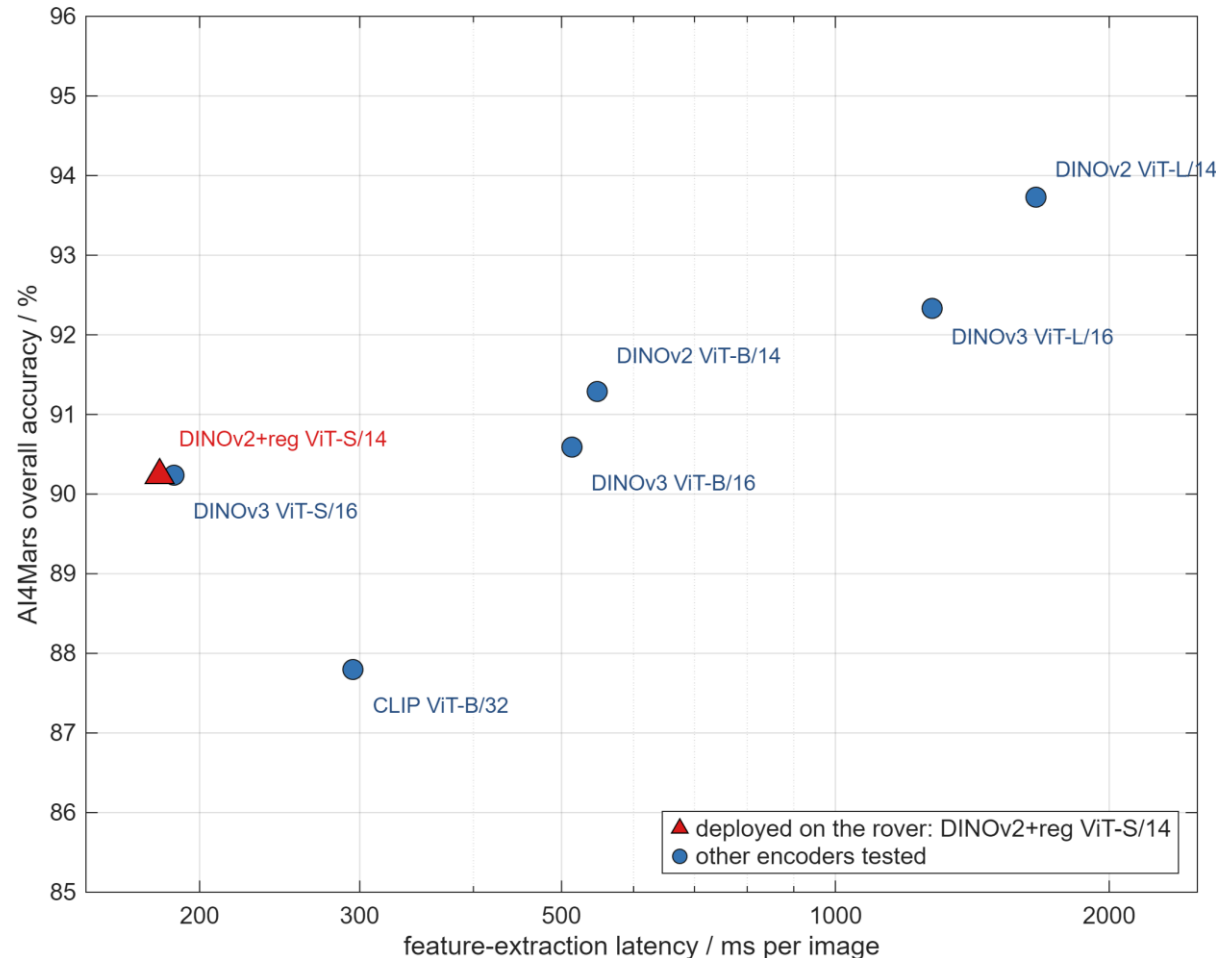
How the rover fails safely when no safe action is available



Red: every route that ends the mission. The rover stops; it never guesses.

When no safe action is available, the system transitions to FAILSAFE and stops the wheels.

Accuracy–latency trade-off: why the smaller model was deployed



The red marker is the deployed model: DINOv2+reg ViT-S/14 at 90.24% accuracy.

It answers in 181 ms. The most accurate model, 93.73%, needs 1,660 ms.

A 3.49-point accuracy loss reduced feature-extraction latency by about 9×.

All encoders were benchmarked on the same machine, so the latency comparison is directly comparable.



PART 4

Simulation

Testing the whole loop in a world where the right answer is known.

A Mars-like Gazebo world to test the system before hardware trials

Zone	Terrain	Correct action	Why it is in the world
1	soil	drive	the easy, common case
2	bedrock	drive slowly	hard ground, safe but rough
3	sand	drive slowly	the class that traps real rovers
4 and 5	big rock	stop	the hazard, sampled twice, 4 m apart

Five test positions over four terrain types, each with a known answer.

Big rock is tested at two positions 4 m apart, because it is the hazard.

The ground uses real AI4Mars textures, so the camera input resembles Mars imagery.

Simple geometry, real Mars textures: the high-detail version was too computationally expensive to run reliably.

Three of five positions classified correctly: five of five decisions were safe

3 / 5

terrain named correctly

soil, bedrock and sand right
both big-rock zones read as sand

5 / 5

safe action taken

the rover stopped at both
rock zones anyway

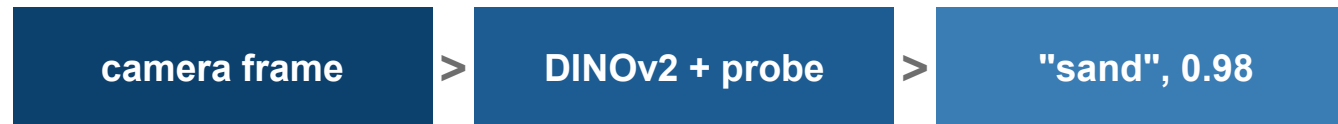
Correct: soil, bedrock, and sand were classified correctly at their test positions.

Incorrect: both big-rock positions were classified as sand.

Yet the rover stopped at both, so no unsafe action was ever taken.

Two scores, because naming the terrain and choosing a safe action are two different things.

The rover stopped safely but not because the foundation model was correct



the classifier is confident, and wrong

In both rock zones the model answers 'sand' confidently, not 'unsure'.

Simulation-only non-learned obstacle gate
Separate non-learned image-processing check

A separate non-learned obstacle check sees the obstacle and stops the rover.



So five of five is the safety net's result, not the model's.

The safe outcome is real. Crediting it to the foundation model would not be.



PART 5

Real-rover experiments

Indoors in a sandpit first, then outdoors, where the surprises were.

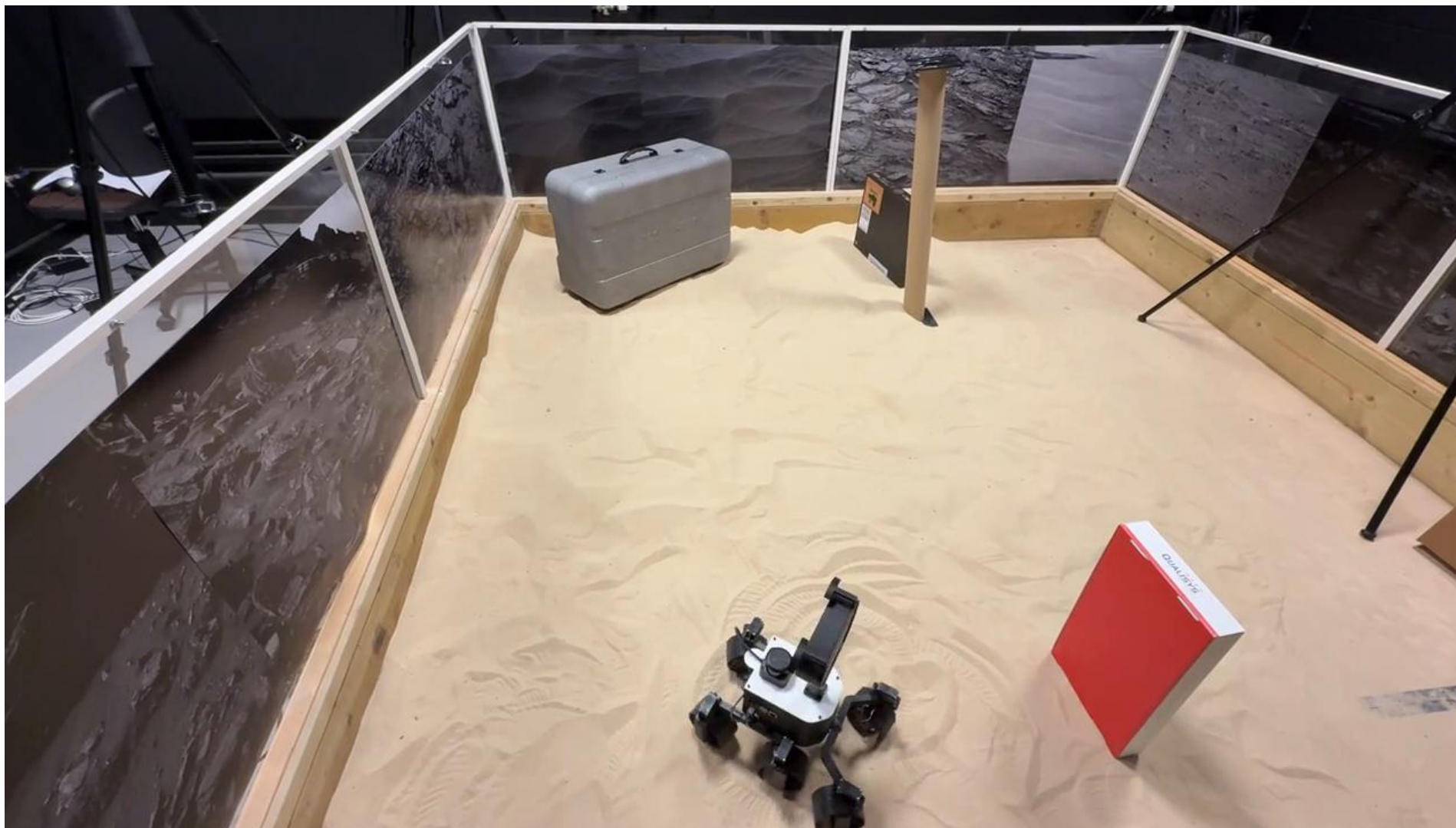
Example run 1: a complete autonomous hardware run



Example run 2: different obstacle layout



Example run 3: another obstacle layout



What the rover sees: onboard terrain classification during operation



Evidence that terrain perception changes the rover's route

1 The geometry-only heading picker selected one direction.

2 The terrain classifier rejected that direction, so the rover turned another way.

3 This behaviour was recorded on the real rover, not in simulation.

Point-turn performance depends on the surface

Surface	Point turn	Outcome
sand	completes	drives on
smooth indoor floor	completes	drives on
dry grass	wheel lifts	run stops
concrete, tarmac	wheel lifts	run stops

On sand and smooth indoor floor the rover turns on the spot exactly as designed.

On dry grass, concrete and tarmac a wheel lifts and the turn never finishes.

Point turns completed on sand and smooth indoor flooring, but failed on several outdoor surfaces when a wheel lifted.

Outdoors, uncertain terrain predictions could stop the rover

The terrain classifier returned “uncertain”, no candidate heading was accepted, and the safety logic stopped the run.

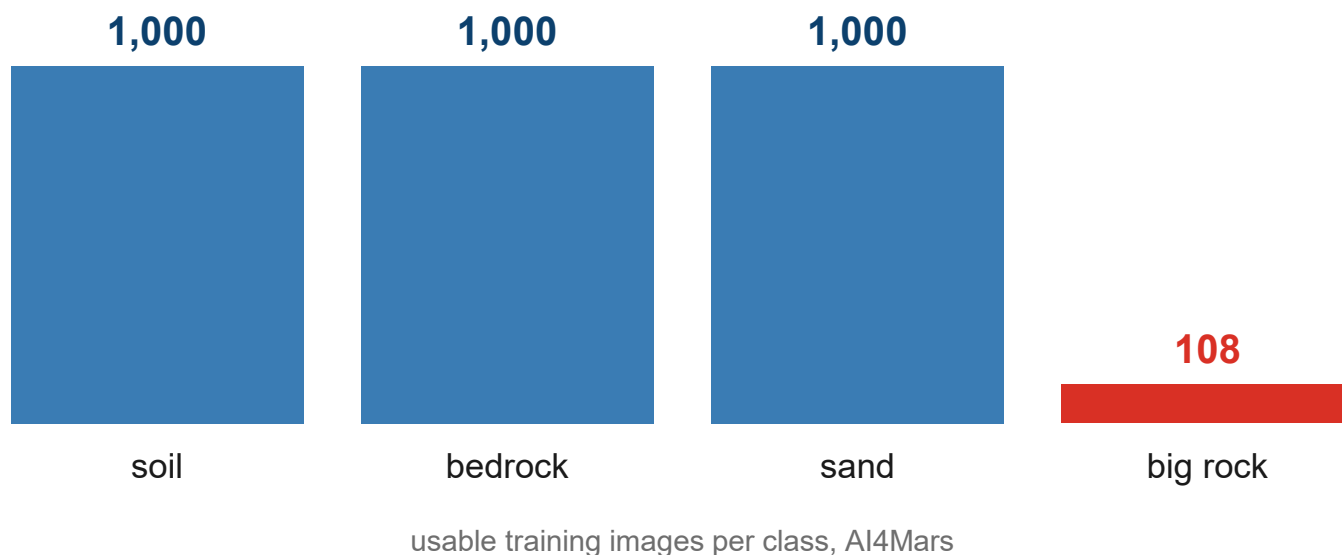


PART 6

Limitations and future work

What it still cannot do, and what would have to happen next.

Limitation 1: the model almost never predicts big rock



The big-rock class is trained on only 108 usable images.

Every other class has roughly 1,000, so it is outnumbered ten to one.

Across the 287-image test set and 36 real-rock photographs, the classifier never predicted the big-rock class.

Because the classifier never selects big rock on the 287-image test set, the big-rock output contributes nothing to the reported headline accuracy.

Limitation 2: the model does not recognise when it is unfamiliar

36

real-rock photographs
tested

0

predicted as big rock or
rejected as uncertain

0.50

mean confidence
above the 0.40 stop
threshold

Future work: one research problem and three engineering tasks

Research problem

A model that knows when it is looking at something it has never seen

Engineering tasks

More big-rock labels, a chassis that can turn reliably across different surfaces a faster runtime

Then measure it again

The outdoor test repeated in daylight, with every sensor logged

Research: a model that can say 'I have not seen this before', and be right.

Engineering: more big-rock labels, more reliable turning across surfaces, and a faster runtime.

Then repeat the outdoor test properly, in daylight, with everything logged.

The first problem matters most: the rover must know when its perception cannot be trusted.

A frozen visual model transferred well enough to Mars terrain to change the rover's route

- The frozen encoder ran onboard a Raspberry Pi 4, with only a lightweight classifier trained on Mars labels.
- The experiments also exposed where that transfer breaks: big rocks, unfamiliar terrain, and real-hardware constraints.



Q & A

Questions

Three faults revealed only by real-hardware testing

Fault	Revealed by	Visible in simulation?
one side of the rover slower	tape measure	no
IMU drops off the I2C bus	running it for an hour	no
slope-safety behaviour incorrect	a real slope	no, it looked correct

One side of the rover ran slower than the other, from a wiring fault.

The tilt sensor intermittently disappears from its bus under load.

A slope-safety behaviour looked correct in simulation and was wrong.

Simulation tests the design. Hardware tests the assumptions the design was built on.

Full experimental protocol

	value
Model variants compared	22
Pretraining objectives	8
Labels per class	1,000 (big rock: 108)
Held-out test set	287-image gold-standard test set
Classifier	logistic regression, $C = 0.316$, seed 42
Encoder weights updated	none, in any experiment

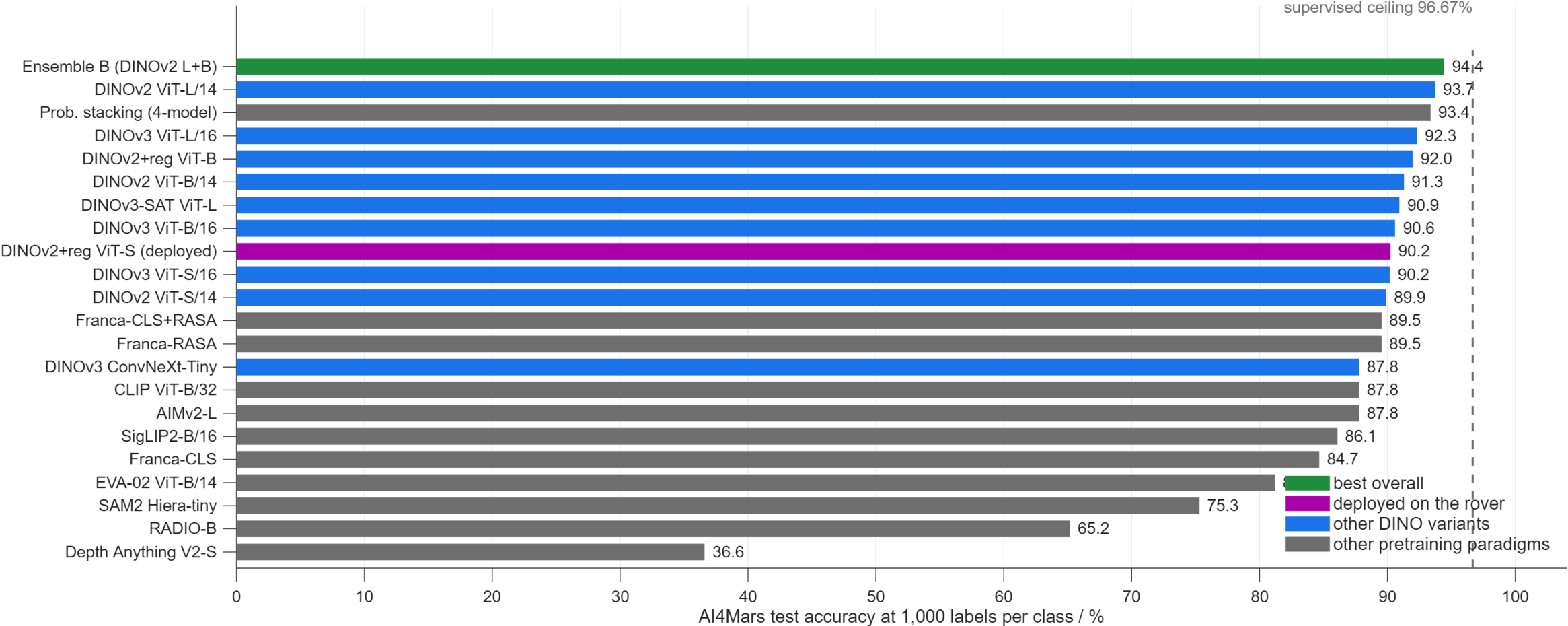
AI4Mars: 1,000 labelled training images per class, with big rock limited to the 108 available images.

Held-out test set of 287 images: 123 soil, 92 bedrock, 72 sand.

Logistic regression, $C = 0.316$, seed 42, identical for all 22 configurations

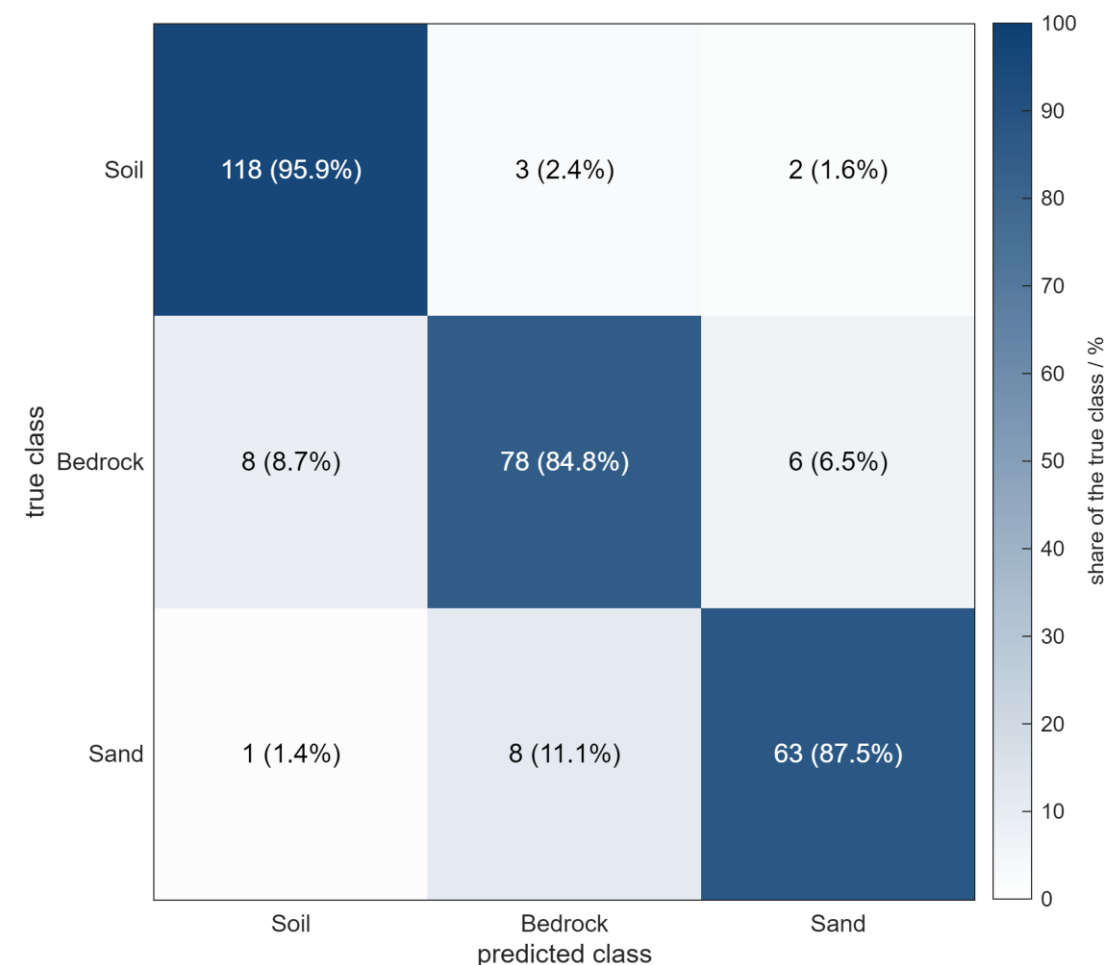
Nothing downstream of the encoder changes between runs. That is what makes the comparison fair.

All 22 model variants, ranked by AI4Mars accuracy



All 22 variants under the same 1,000-shot protocol and held-out test set.

Confusion matrix of the deployed model



Deployed DINOv2+reg ViT-S/14 on the 287-image held-out test set: 90.24% overall accuracy.

Why the reported latency numbers differ

what was measured	latency	what else was running
Development PC	181 ms	nothing
Raspberry Pi, encoder alone	570 ms	nothing
Raspberry Pi, compressed	386 ms	nothing
Raspberry Pi, whole rover running	1,865-2,200 ms	13 programs, 4 cores
Measured outdoors on the rover	1,550-1,600 ms	the deployed system

The encoder measured alone is not what the rover experiences.

Compared with the compressed encoder alone, running the full rover stack increases latency by roughly 5×.

The field test measured the deployed system directly and confirmed the same order of full-stack latency.

Latency depends strongly on what is measured and what else is running concurrently.

Model compression introduces an accuracy trade-off

90.24%

the deployed model in full
precision

86.41%

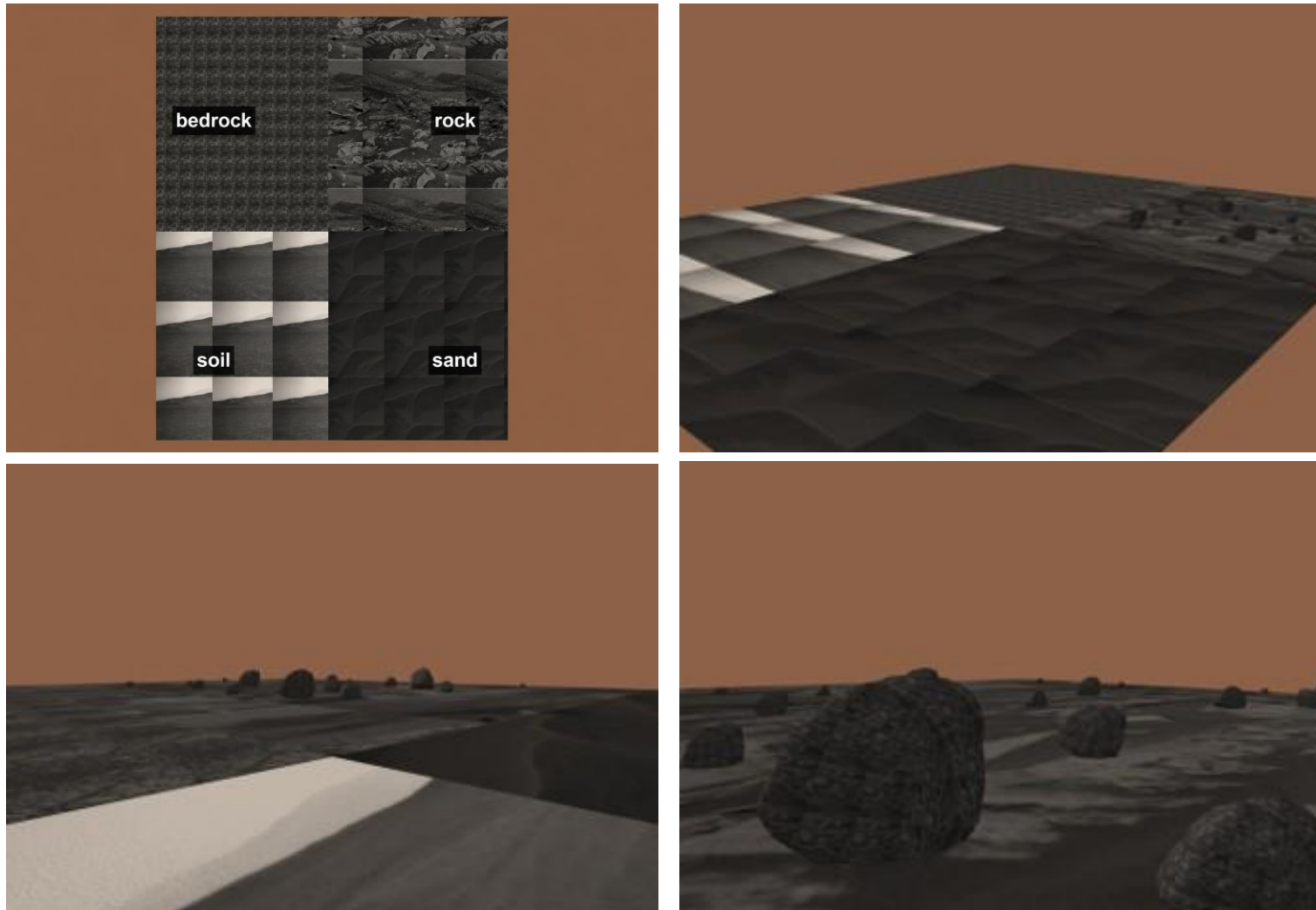
the same encoder
compressed to 8-bit
integers

3.83

percentage points lost,
concentrated in the soil
class

The latency improved, but the accuracy loss was too large to justify deployment.

Mars-like Gazebo world used for system testing



Real AI4Mars textures were applied to simple Gazebo geometry to create controlled terrain and hazard test zones.